

lab2_report

: 2022056562 서민용, 2023003227 이한주

1. Overall structure

```
module RF (  
    // You may also change the  
    input and output ports (maybe c  
    hanging reg to wire)  
    input clk,  
    input rst,  
    // Read-related ports  
    input [4:0] rd_addr1,  
    input [4:0] rd_addr2,  
    output reg [31:0] rd_da  
    ta1,  
    output reg [31:0] rd_da  
    ta2,  
    // Write-related ports  
    input RegWrite,  
    input [4:0] wr_addr,  
    input [31:0] wr_data  
);
```

```
module ALU(  
    input [31:0] ope  
    rand1,  
    input [31:0] ope  
    rand2,  
    input [4:0] sha  
    mt,  
    input [3:0] fun  
    ct,  
    output reg [31:0] alu  
    _result  
);
```

```
module TOP(  
    input clk,  
    input rst,  
  
    // Read-related ports  
    input [4:0] rd_addr1,  
    input [4:0] rd_addr2,  
  
    // Write-related ports  
    input RegWrite,  
    input [4:0] wr_addr,  
  
    // ALU ctrl and output  
    input [4:0] shamt,  
    input [3:0] funct,  
    output [31:0] rd_data1,  
    output [31:0] rd_data2,
```

```
output [31:0] wr_data
);
```

- HW1에서 구현했던 ALU와 더불어 함께 작동하는 RF 모듈을 만들고, 이를 TOP 모듈에서 연결하는 코드를 씁니다.
- `rd_data1,2` 는 `rd_addr1,2` 의 값이 변할 때마다 값을 해당 RF 주소에서 읽어옵니다.
- RF 에서 받아온 값은 ALU 모듈의 input이 되어 `operand1,2` 로 `funct` 와 `shamt` 에 따라서 알맞는 연산을 하게 됩니다.
- 해당 연산의 결과 값은 `alu_result` 라는 output으로 나오게 됩니다.
 - 또한, RF에서는 `clk posedge` 이고 `RegWrite` 가 1일 경우 `wr_addr` 에 `wr_data` 를 씁니다.
 - `wr_data` 는 ALU 모듈의 `alu_result` 값으로, ALU 모듈의 output과 RF 모듈의 input은 서로 연결되어야 합니다.

2. Implementation

- `ALU.v` 는 기존 HW1 과 동일하게 작성하였고, `RF.v` , `TOP.v` 는 다음과 같습니다.

```
// RF.v
// ...
reg [31:0] register_file [0:31];

always @(rd_addr1 , rd_addr2) begin
    rd_data1 = register_file[rd_addr1];
    rd_data2 = register_file[rd_addr2];
end
// there are no wires to connect.
//assign ??

always @(posedge clk) begin
    if (rst) begin
        $readmemh("initial_reg.mem", register_file);
    end
    if(RegWrite) begin
        register_file[w_raddr] <= wr_data;
    end
end
```

```
// TOP.v
// ...
// Instantiate modules and connect them!
wire [31:0] net1;
wire [31:0] net2;
wire [31:0] net3;
assign rd_data1 = net1;
assign rd_data2 = net2;
assign wr_data = net3;
// connecting them explicitly.
RF rf (
    .clk(clk),
    .rst(rst),
    .rd_addr1(rd_addr1),
    .rd_addr2(rd_addr2),
    .rd_data1(net1),
    .rd_data2(net2),
    .RegWrite(RegWrite),
    .wr_addr(wr_addr),
    .wr_data(net3)
);
ALU alu(
    .operand1(net1),
    .operand2(net2),
```

```
end
end
```

- `rd_addr` 값이 변했을 때 `rd_data` 값들을 RF에서 받아옵니다.
- 매 `posedge clk` 마다,
 - `rst` 가 1이라면, 초기 값으로 RF를 채웁니다.
 - `RegWrite` 가 1이라면, 현재 `wr_addr` 과 `wr_data` 값으로 RF를 업데이트 해줍니다.

```
.shamt(shamt),
.funct(funct),
.alu_result(net3)
);
```

- `rd_data` 나 `wr_data` 를 직접 연결해도 문법적으로는 문제가 없으나, RF module의 output → ALU module의 input으로 들어가게 되는 구조에서 wire를 거치지 않는게 직관적으로 어색하다고 생각해 `net` 변수들을 추가했습니다.
- 특히 `alu_result` 와 `wr_data` 의 경우, ALU module에서 계산된 값이 `wr_data` 로써 RF module의 input으로 값이 쓰이는 것인데, 직접 연결하게 되면 직관적으로 TOP module의 흐름이 보이지 않는다고 생각했습니다.

3. results & Troubleshooting

- 모든 test 를 다 통과한 모습입니다.

```
minyongseo@Minui-MacBookAir HW02_MinYong % iverilog TOP.v TOP_tb.v ALU.v RF.v
minyongseo@Minui-MacBookAir HW02_MinYong % ./a.out
VCD info: dumpfile waveform.vcd opened for output.
PASSED = 10000, FAILED = 0
TOP_tb.v:129: $finish called at 100035000 (1ps)
minyongseo@Minui-MacBookAir HW02_MinYong %
```

터미널에 출력되는 텍스트 양을 줄이기 위해 testbench 내 `$display` 코드를 일부 주석처리 하였습니다.

- 또한 `TOP_tb.v` 파일에 `waveform.vcd` 를 만들도록 코드를 추가한 뒤, GTKWave를 통해 결과를 자세히 볼 수 있었습니다.
- 디버깅 과정에서, testbench에서의 test는 통과하지만, 내부적으로는 원치 않는 동작이 일어나는 것을 발견했습니다.
 - 물론 현재 testbench에서 각 테스트케이스 별로 단 한번의 `posedge clk`만 주기 때문에, `register_file` 에 원치 않는 값을 적히게 되는 일은 없습니다.
- `RF.v` 에서, `rd_data` 들에 대해서 asynchronous하게 동작하는 코드를 짤 때, `@(*)` 을 사용해서 생긴 문제였습니다.
- 수업시간에 배운 내용으로, `*` 은 해당 always block 내 모든 righthand side에 있는 모든 변수들의 값에 변화가 있을 때 동작하게 하는데, `rd_data` 에 값을 불러오게 되는 시점에 `rd_addr` 뿐만 아니라, `register_file` 역시 영향을 미친다는 것을 파악했습니다.

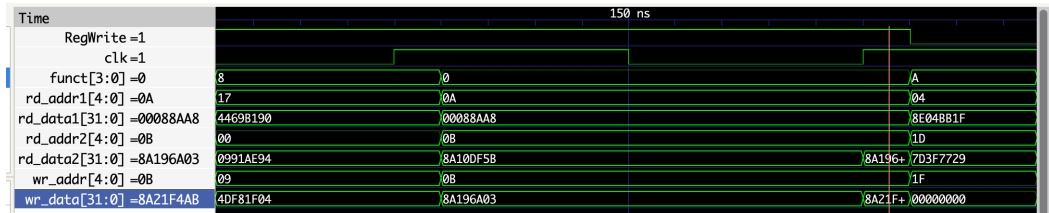
```
always @(*) begin
    rd_data1 = register_file[rd_addr1];
```

```

rd_data2 = register_file[rd_addr2];
end

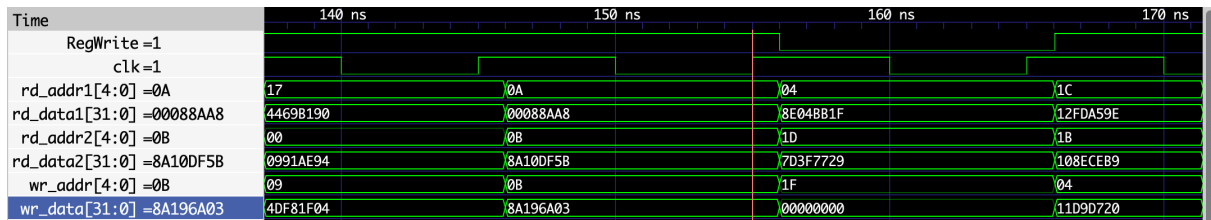
```

- `register_file` 은 매 `posedge clk` 와 `RegWrite` 일 때마다 변화가 생기는데, 이 경우에도 `rd_data` 를 업데이트 하게 됩니다.
 - 여기서 문제상황은 `rd_addr` 과 `wr_addr` 이 같은 주소를 가지고 `RegWrite` 가 1일 때입니다.



inaccurate version [always @(*) ...]

- 주소 값이 바뀐 시점에서 올바른 계산이 되어 `wr_data` 값은 적절한 값이고, 이 값이 다음 `posedge clk` 일 때 `RegWrite` 가 1이라면 이 값을 `register_file` 의 `wr_addr` 에 쓰게 됩니다.
 - 하지만 `@(*)` 으로 작성할 경우, `register_file` 의 `wr_addr` : 0B 에 값을 쓴 직후, `rd_addr2` 역시 0B 이기 때문에 `@(*)` 에 감지되어 다시 한번 그 값을 `rd_data` 에 불러오게 됩니다.
 - 그 이후 만일 다시 `posedge clk` 일 때 `RegWrite` 가 1이 되는 상황이 온다면, 원치 않는 값이 `register_file` 에 적히게 됩니다.
- 하여 `@(*)` 이 아니라 `@(rd_addr1, rd_addr2)` 라고 명시해주어야 강건합니다.



correct version [always @(rd_addr1, rd_addr2) ...]